

Documentação do Sistema

Gerador de Relatórios Google Ads — Tráfega Mídia

Versão 1.0.0 · Documento técnico e de uso · Gerado em 15/08/2026

1. Visão geral

Interface web que lê **2 prints do Google Ads** com a API da OpenAI (visão), extrai os números e gera automaticamente um **PDF de relatório mensal** no padrão da Tráfega Mídia — com template, logo e layout fixos. O objetivo é fechar cada relatório de cliente em **menos de 1 minuto**, sem edição manual de documento.

Fluxo por cliente: seleciona o cliente → sobe os 2 prints → a IA extrai os dados → confere/ajusta os campos → clica em **Gerar Relatório** → baixa o PDF.

Para quem é

Equipe de tráfego da agência, que emite dezenas de relatórios por mês.

Entrada

2 imagens: visão geral (cards de performance) + tabela de leilão.

Saída

PDF A4 Relatório <Mês> - <Cliente>.pdf

IA

Só a leitura dos prints usa a API; o PDF é montado localmente.

2. Arquitetura

Aplicação **Node.js + Express**, frontend em HTML/CSS/JS puro (sem build). Roda tanto local quanto **serverless na Vercel** (mesma base de código).

Camada	Tecnologia	Arquivo
Servidor / API	Express 5	server.js
Entrada serverless	Vercel Functions	api/index.js + vercel.json
Leitura dos prints (IA)	OpenAI API — visão + JSON Schema strict	lib/extract.js
Montagem do relatório	Gerador de HTML/CSS/SVG	lib/report.js
HTML → PDF	Puppeteer + Chrome/Chromium	lib/pdf.js
Config (chave + modelo)	JSON em disco / env vars	lib/settings.js
Interface	HTML/CSS/JS puro	public/
Template do relatório	Logo + head HTML	assets/
Dados persistidos	JSON local	data/clients.json , data/settings.json

Fluxo de dados

```
Prints (PNG/JPG)
| POST /api/extract (multipart, 2 imagens)
▼
lib/extract.js — OpenAI (visão, json_schema strict) → JSON estruturado
| usuário confere/edita no navegador
▼
POST /api/generate (JSON do relatório)
|
```

```
lib/report.js → HTML + gráfico SVG + tabela de leilão  
|  
lib/pdf.js → Puppeteer renderiza A4 → PDF (download)
```

3. Rotas da API

Rota	Método	Função
/api/settings	GET	Estado da configuração: se há chave, chave mascarada, origem (env/disco), se o disco é só-leitura e o modelo atual.
/api/settings	POST	Salva a chave da OpenAI e/ou o modelo escolhido.
/api/settings/key	DELETE	Remove a chave salva (se houver chave no <code>.env</code> , ela continua valendo).
/api/models	GET	Lista os modelos da conta OpenAI do usuário (fallback embutido se falhar).
/api/clients	GET	Lista os clientes salvos (nome, gestor, preferência).
/api/clients	POST	Cria/atualiza um cliente por nome.
/api/clients/:nome	DELETE	Remove um cliente.
/api/extract	POST	Recebe os 2 prints e devolve os dados estruturados do relatório.
/api/identify	POST	Identifica 1 print (nome do cliente + tipo) — usado na geração em massa para agrupar prints.
/api/generate	POST	Recebe o JSON do relatório e devolve o PDF pronto para download.
/api/preview	POST	Devolve o HTML do relatório (pré-visualização sem baixar o PDF).
/api/health	GET	Healthcheck: <code>{ ok, hasKey, model }</code> .

4. A leitura por IA (extração)

O módulo `lib/extract.js` envia as 2 imagens (em base64, `detail: high`) para a OpenAI com um **system prompt** que codifica as regras do relatório da Tráfega, e força a resposta via **JSON Schema strict** — garantindo saída sempre parseável e no formato certo.

O schema padroniza:

- **periodo** — período exato como no print.
- **metricas** — sempre 4 cards (Conversões Totais · Custo por Conversão · CPC Médio · Investimento). Se o cliente marca "sem custo por conversão", o 2º card vira **Cliques**.
- **grafico** — conversões por semana (o total das semanas soma o total do mês).
- **leilao** — todos os concorrentes na mesma ordem; a linha do cliente vem destacada (`ehVoce=true`).
- **passos** — 4 próximos passos baseados nos números reais.
- **avisos** — ressalvas de transparência (ex.: gráfico estimado, valor calculado).

5. Configuração & execução

Rodar localmente

```
npm install      # instala dependências (Puppeteer usa o Chrome da máquina)
npm start       # sobe em http://localhost:3000
npm run dev     # modo watch (reinicia ao salvar)
```

Variáveis de ambiente

OPENAI_API_KEY	Chave da OpenAI. Alternativa: salvar pela interface (tem prioridade sobre a env).
OPENAI_MODEL	Modelo padrão (default <code>gpt-5.4-mini</code>).
PORT	Porta local (default 3000).
PUPPETEER_EXECUTABLE_PATH	Caminho manual do Chrome, se não for autodetectado.

A chave também pode ser colada direto no painel **Configurações da IA**. Ela fica em `data/settings.json` (fora do git) e tem prioridade sobre o `.env`.

Deploy na Vercel

O projeto já está preparado para serverless: `api/index.js` reaproveita o app Express e `vercel.json` reescreve todas as rotas para essa função, com `maxDuration` de 60s. Diferenças no ambiente Vercel (disco somente-leitura):

- A chave vem da env var `OPENAI_API_KEY` (o campo na interface fica só-leitura).
- Modelo e lista de clientes ficam no **navegador** (localStorage), não no servidor.
- O PDF é gerado com **Chromium serverless** (`@sparticuz/chromium`) em vez do Chrome local.

6. Como usar (passo a passo)

- 1 Cliente & período** — escolha um cliente salvo (ou **+ Novo cliente**), confira mês/ano e gestor. Marque "sem custo por conversão" se o cliente preferir o card de Cliques.
- 2 Prints** — arraste os 2 prints (visão geral + tabela de leilão). A ordem não importa.
- 3 Ler prints com a IA** — a OpenAI lê as imagens e preenche todos os campos.
- 4 Conferir** — todos os campos são editáveis; avisos de transparência aparecem em destaque. Use  **Pré-visualizar** para ver antes de baixar.
- 5 Gerar Relatório (PDF)** — baixa `Relatório <Mês> - <Cliente>.pdf`.

7. Dependências principais

Pacote	Uso
<code>express</code>	Servidor HTTP e rotas da API.
<code>openai</code>	Cliente da API de visão (leitura dos prints).
<code>multer</code>	Upload das imagens (em memória, até 15 MB por print).
<code>puppeteer-core</code>	Renderiza o HTML do relatório em PDF.
<code>@sparticuz/chromium</code>	Chromium para o ambiente serverless da Vercel.
<code>dotenv</code>	Carrega variáveis do <code>.env</code> .

